

Ejercicio 12

Resuelva la función de tiempo de ejecución de los siguientes segmentos de código y calcule el orden.

<pre>int recur (int n){ if (n == 1) return 1; else return (recur(n/2) * recur(n/2)); }</pre>	<pre>int recursivo(int n){ if (n <= 5) return 1; else return (recursivo (n-5)); }</pre>
<pre>int recursivo(int n){ if (n <= 7) return 1; else return (recursivo (n/8)); }</pre>	

recur

Por un lado tenemos una operación constante para los valores de $n \leq 1$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realizan dos llamadas recursivas enviando como parámetro $\frac{n}{2}$:

$$c_2 + 2 \cdot T\left(\frac{n}{2}\right)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n = 1, \\ c_2 + 2 \cdot T\left(\frac{n}{2}\right) & n > 1. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + 2 \cdot T\left(\frac{n}{2}\right)$
2. $T(n) = c_2 + 2[c_2 + 2T\left(\frac{n}{2 \cdot 2}\right)] = 3c_2 + 2 \cdot 2T\left(\frac{n}{2}\right) = 3c_2 + 2^2T\left(\frac{n}{2^2}\right)$
3. $3c_2 + 2^2[c_2 + 2T\left(\frac{n}{2^2 \cdot 2}\right)] = 7 \cdot c_2 + 2^3 \cdot T\left(\frac{n}{2^3}\right)$

Paso i :

$$T(n) = (2^i - 1) \cdot c_2 + 2^i \cdot T\left(\frac{n}{2^i}\right)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n = 1$, vamos a tomar el 1.
O sea,

$$\begin{aligned}1 &= \frac{n}{2^i} \\ 2^i &= n \\ i &= \log_2(n)\end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$\begin{aligned}T(n) &= (2^{\log_2 n} - 1) \cdot c_2 + 2^{\log_2 n} \cdot T\left(\frac{n}{2^{\log_2 n}}\right) \\ &= (n - 1) \cdot c_2 + n \cdot T(1) \\ &= c_2 \cdot n - c_2 + n \cdot c_1 \\ \text{Renombramos, } c_3 &= -c_2 \text{ y } c_4 = c_1 + c_2 \\ &= (c_1 + c_2) \cdot n + c_3 \\ &= c_4 \cdot n + c_3\end{aligned}$$

Entonces, la función de tiempo de ejecución de rec1 es

$$\boxed{T(n) = c_4 \cdot n + c_3}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = c_4 \cdot n + c_3 \leq O(n)}$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_4 \cdot n + c_3 \leq c \cdot n, \text{ para todo } n \geq n_0$$

- Primer término, $c_4 \cdot n$
Se cumple que $n \leq n$. Si multiplicamos a ambos lados por c_4 .

$$c_4 \cdot n \leq c_4 \cdot n$$

Con esto hallamos el valor de k_1 , es $k_1 = c_4$. Esta desigualdad vale para $n_0 = 0$.

- Segundo término, c_3
Se cumple que $1 \leq n$.
Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3 n$$

Hallamos el valor de k_2 , $k_2 = c_3$. La desigualdad se cumple para los $n \geq 1$.
Tomamos $n_0 = 1$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$\begin{aligned}
 c_4 \cdot n + c_3 &\leq k_1 \cdot n + k_2 \cdot n \\
 T(n) &\leq (k_1 + k_2) \cdot n \\
 T(n) &\leq (c_4 + c_3) \cdot n \\
 \text{Renombrando } c &= c_4 + c_3 \\
 T(n) &\leq c \cdot n
 \end{aligned}$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_4 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

recursivo

```

int recursivo(int n){
    if (n <= 5)
        return 1;
    else
        return (recursivo (n-5));
}

```

Por un lado tenemos una operación constante para los valores de $n \leq 5$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realiza una llamada recursiva enviando como parámetro $n - 5$:

$$c_2 + T(n - 5)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 5, \\ c_2 + T(n - 5) & n > 5. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + T(n - 5)$
2. $T(n) = c_2 + (c_2 + T(n - 10)) = 2 \cdot c_2 + T(n - 10)$
3. $T(n) = 2c_2 + (c_2 + T(n - 15)) = 3c_2 + T(n - 15)$
- $\vdots$

Paso i :

$$T(n) = i \cdot c_2 + T(n - 5i)$$

Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 5$, vamos a tomar el 1.

O sea,

$$\begin{aligned}n - 5i &= 5 \\ -5i &= 5 - n \\ i &= \frac{n - 5}{5}\end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.

$$\begin{aligned}T(n) &= \left(\frac{n - 5}{5}\right) \cdot c_2 + T\left(n - 5 \cdot \left(\frac{n - 5}{5}\right)\right) \\ &= \frac{c_2}{5}n - c_2 + c_1\end{aligned}$$

$$\begin{aligned}\text{Renombramos } c_3 &= c_1 - c_2 \text{ y } c_4 = \frac{c_2}{5} \\ &= c_4n + c_3\end{aligned}$$

Entonces, la función de tiempo de ejecución de `rec1` es

$$T(n) = c_4 \cdot n + c_3$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$T(n) = c_4 \cdot n + c_3 \leq O(n)$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_4 \cdot n + c_3 \leq c \cdot n, \text{ para todo } n \geq n_0$$

- Primer término, $c_4 \cdot n$

Se cumple que $n \leq n$. Si multiplicamos a ambos lados por c_4 .

$$c_4 \cdot n \leq c_4 \cdot n$$

Con esto hallamos el valor de k_1 , es $k_1 = c_4$. Esta desigualdad vale para $n_0 = 0$.

- Segundo término, c_3

Se cumple que $1 \leq n$.

Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3n$$

Hallamos el valor de k_2 , $k_2 = c_3$. La desigualdad se cumple para los $n \geq 1$. Tomamos $n_0 = 1$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$\begin{aligned}
 c_4 \cdot n + c_3 &\leq k_1 \cdot n + k_2 \cdot n \\
 T(n) &\leq (k_1 + k_2) \cdot n \\
 T(n) &\leq (c_4 + c_3) \cdot n \\
 \text{Renombrando } c = c_4 + c_3 \\
 T(n) &\leq c \cdot n
 \end{aligned}$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_4 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

recursivo

```

int recursivo(int n){
    if (n <= 7)
        return 1;
    else
        return (recursivo (n/8));
}

```

Por un lado tenemos una operación constante para los valores de $n \leq 7$ que vamos a representar con la constante c_1 .

Por otro lado, para el resto de valores de n , vemos que se realiza una llamada recursiva enviando como parámetro $\frac{n}{8}$:

$$c_2 + T\left(\frac{n}{8}\right)$$

Podemos expresar a $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 7, \\ c_2 + T\left(\frac{n}{8}\right) & n > 7. \end{cases}$$

Analizamos el desarrollo de la llamada recursiva para encontrar el paso general.

1. $T(n) = c_2 + T\left(\frac{n}{8}\right)$
2. $T(n) = c_2 + (c_2 + T\left(\frac{n}{8^2}\right)) = 2c_2 + T\left(\frac{n}{8^2}\right)$
3. $T(n) = 2c_2 + (c_2 + T\left(\frac{n}{8^3}\right)) = 3c_2 + T\left(\frac{n}{8^3}\right)$

Paso i , $T(n) = ic_2 + T\left(\frac{n}{8^i}\right)$ Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 7$, vamos a tomar el 1.
O sea,

$$\begin{aligned} 7 &= \frac{n}{8^i} \\ 8^i &= \frac{n}{7} \\ i &= \log_8\left(\frac{n}{7}\right) \end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general.
Propiedades:

- $a^{\log_a(b)} = b$
- $\log_a\left(\frac{b}{c}\right) = \log_a b - \log_a c$

$$\begin{aligned} T(n) &= \log_8\left(\frac{n}{7}\right)c_2 + T\left(\frac{n}{8^{\log_8(\frac{n}{7})}}\right) \\ &= \log_8\left(\frac{n}{7}\right)c_2 + c_1 \\ &= \log_8(n)c_2 - \log_8(7)c_2 + c_1 \\ \text{Renombrando } c_3 &= c_1 - \log_8(7)c_2 \\ &= \log_8(n)c_2 + c_3 \end{aligned}$$

Entonces, la función de tiempo de ejecución de rec1 es

$$\boxed{T(n) = \log_8(n)c_2 + c_3}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = \log_8(n)c_2 + c_3 \leq O(\log_8(n))}$$

Hay que buscar los valores constantes c y n_0 tales que

$$\log_8(n)c_2 + c_3, \text{ para todo } n \geq n_0$$

Analizamos término a término

- Primer término, $\log_8 n \cdot c_2$
Se cumple que $\log_8 n \leq \log_8 n$.
Si multiplicamos a ambos lados por c_2 .

$$c_2 \log_8 n \leq c_2 \log_8 n$$

Con esto hallamos el valor de $k_1 = c_2$. Esta desigualdad vale para $n_0 = 1$.
El logaritmo no está definido para $n \geq 1$.

- Segundo término, c_3
Se cumple que $1 \leq c_3$. Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3 \log_8 n$$

Con esto hallamos el valor de $k_2 = c_3$. Esta desigualdad vale para $n_0 = 1$.
El logaritmo no está definido para $n \geq 1$.

Ahora sumamos a cada lado, los resultados parciales que obtuvimos.

$$\log_8(n)c_2 + c_3 \leq k_1 \log_8 n + k_2 \log_8 n$$

$$T(n) \leq (k_1 + k_2) \log_8 n$$

$$T(n) \leq (c_2 + c_3) \log_8 n$$

Renombramos $c = c_2 + c_3$

$$T(n) \leq c \cdot \log_8 n$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(\log_8 n), \text{ con } c = c_2 + c_3 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$